

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	B.Vani	Reg. No.:	192472348
Section / Batch:	2024 CSE AI	Date:	11-07-2026

Code of Conduct

I B.Vani (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.Vani

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number

```
Python 3.10.11 - C:/Users/rishi/Downloads/quesinp.py (3.10.11)
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5e, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
>>>
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/rishi/Downloads/quesinp.py =====
Student ID: ['101']
Email: ['john.doe123@gmail.com']
Mobile: ['+91-9876543210']
PIN Code: ['560078']
Password: ['P@ssw0rd123']
Date of Birth: ['15/08/2004']
Register Number: ['23AIML1056']
>>>

Python 3.10.11 - C:/Users/rishi/Downloads/quesinp.py (3.10.11)
File Edit Format Run Options Window Help
import re

text = """
Student ID: 101
Name: John Doe
Email: john.doe123@gmail.com
Mobile: +91-9876543210
PIN: 560078
Password: P@ssw0rd123
Date of Birth: 15/08/2004
Register Number: 23AIML1056
Department: Artificial Intelligence and Machine Learning
"""

print("Student ID:", re.findall(r'\b\d{3}\b', text))
print("Email:", re.findall(r'[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}', text))
print("Mobile:", re.findall(r'\d{10}', text))
print("PIN Code:", re.findall(r'\d{6}', text))
print("Password:", re.findall(r'\b[a-zA-Z0-9@!#$%^&*]{8,}', re.search(r'Password:.*', text).group(1)))
print("Date of Birth:", re.findall(r'\d{2}/\d{2}/\d{4}', text))
print("Register Number:", re.findall(r'\d{2}[A-Z]{4}\d{4}', text))
```

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

6.

```
Python 3.10.11 (tags/v3.10.11:744cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/rishi/Downloads/ques22.py =====
Enter the word to search: madam

Occurrences: 0
The word is NOT found.

Positions:
Paragraph after replacement:
Artificial Intelligence (AI) is transforming industries across the world.
AI is used in healthcare to assist doctors in diagnosis.
In banking AI detects fraud and improves customer experience.
Many companies invest heavily in AI research because AI improves efficiency
and enables intelligent decision making.
```

```
ques22.py - C:/Users/rishi/Downloads/ques22.py (3.10.11)
File Edit Format Run Options Window Help
import re

paragraph = """
Artificial Intelligence (AI) is transforming industries across the world.
AI is used in healthcare to assist doctors in diagnosis.
In banking AI detects fraud and improves customer experience.
Many companies invest heavily in AI research because AI improves efficiency
and enables intelligent decision making.
"""

word = input("Enter the word to search: ")

matches = re.findall(word, paragraph, re.IGNORECASE)

print("\nOccurrences:", len(matches))

if matches:
    print("The word is found.")
else:
    print("The word is NOT found.")

print("\nPositions:")

for match in re.finditer(word, paragraph, re.IGNORECASE):
    print(match.start())

print("\nParagraph after replacement:")
print(re.sub(word, "AR", paragraph, flags=re.IGNORECASE))
```

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

```
Python 3.10.11 (tags/v3.10.11:744cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/rishi/Downloads/ques3.py =====
Sentence Split:
Artificial Intelligence is changing the world
It helps in healthcare,
banking, education and transportation
Python is one of the popular
programming languages
It is simple and powerful

Word Split:
['Artificial', 'Intelligence', 'is', 'changing', 'the', 'world', 'It', 'helps', 'in', 'h
ealthcare', 'banking', 'education', 'and', 'transportation', 'Python', 'is', 'one', 'of',
'the', 'popular', 'programming', 'languages', 'It', 'is', 'simple', 'and', 'powerful']

Total Words: 27
>>>
```

```
ques3.py - C:/Users/rishi/Downloads/ques3.py (3.10.11)
File Edit Format Run Options Window Help
import re

paragraph = """
Artificial Intelligence is changing the world. It helps in healthcare,
banking, education and transportation! Python is one of the popular
programming languages? It is simple and powerful.
"""

print("Sentence Split:")
sentences = re.split(r'[.!?]+', paragraph)

for s in sentences:
    if s.strip():
        print(s.strip())

print("\nWord Split:")

words = re.findall(r'\b\w+\b', paragraph)

print(words)

print("\nTotal Words:", len(words))
```

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
File Edit Format Run Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/cish1/downloads/ques4.py =====
Enter Email: cishithomas0951@gmail.com
Enter Password: cish1223
Valid Email
Weak Password

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
File Edit Format Run Options Window Help
import re

email = input("Enter Email: ")
password = input("Enter Password: ")

email_pattern = r'^(?!(\d+)|[^\w\d@+]{4,})[a-zA-Z\d+@-]{1,32}$'
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@_!#$%^&*()~`{|}~']).{8,}$'

if re.match(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.match(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
```

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the `re.compile()` method to compile the pattern once and reuse it to validate all the email IDs.

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
File Edit Shell Debug Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/cish1/downloads/ques5.py =====
john@gmail.com -> Valid
ebc@yahoo.in -> Valid
hello123@outlook.com -> Valid
wrongemail.com -> Invalid
test@.com -> Invalid
student2026@gmail.com -> Valid
>>>

Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD64)]
File Edit Format Run Options Window Help
import re

emails = [
    "john@gmail.com",
    "ebc@yahoo.in",
    "hello123@outlook.com",
    "wrongemail.com",
    "test@.com",
    "student2026@gmail.com"
]

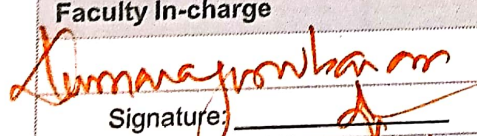
pattern = re.compile(r'^(?!(\d+)|[^\w\d@+]{4,})[a-zA-Z\d+@-]{1,32}$')

for email in emails:
    if pattern.match(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1	8	8	8	8	8	20	100
2	8	8	8	8	8	20	
3	8	8	8	8	8	20	
4	8	8	8	8	8	20	
5	8	8	8	8	8	20	

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: <u>22/04/21</u>	Date: _____	Date: _____